COP3530 Project 2

MiniDB

November 2026

Administrative Information

Team name	Oracle (Group 94)
Team members	Zhengmao Zhang (billzhengmaozhang) Jacob Lindland (jdljake) My Pham (phammy237)
GitHub repository	billzhengmaozhang/COP3530p2
Video demo	Link to be added when the recording is ready

1. Extended and Refined Proposal

Problem and motivation. MiniDB is an in-memory database for stock trades. It answers exact-ID queries and compares two ways of finding the same trade: a custom separate-chaining hash table and a custom B+ tree. Searching an unsorted collection can require inspecting every row. An index avoids a full record scan, but it takes work to build and maintain. The project asks how much that work costs and whether the answer changes with dataset size, insertion order, or whether a queried ID exists.

Implemented features. The terminal menu loads and saves trade CSV files, inserts a validated trade, and looks up a trade by ID using the index selected by the user. Successful insertion updates both indexes. Duplicate IDs are rejected and a failed CSV load preserves the active dataset. A successful load replaces the active dataset. If an insertion fails part-way for an unexpected reason such as an allocation failure, the database is rebuilt from the previously committed records so the record store and both indexes never disagree. The benchmark menu generates synthetic data, compares the two indexes in isolation, reports progress, prints median timings, and saves per-trial results as CSV.

Data. Each row has a positive 64-bit trade ID, a ticker symbol, a price in integer cents, a positive share quantity, and a nonnegative Unix timestamp in seconds. Storing prices in cents preserves their exact values. The generator assigns unique IDs at a configurable stride and draws the remaining fields from fixed distributions using a seed. It can emit rows in ascending ID order or shuffle those same rows. CSV input may use CRLF line endings, blank lines, and a UTF-8 byte order mark. The repository includes a small synthetic sample CSV. The default benchmark generates 100,000, 250,000, and 500,000 trades in both orders, using odd IDs only (stride 2) so that absent IDs exist between the stored ones.

Tools and data structures. MiniDB uses C++17 and CMake. Catch2 and CTest run the test suite, which can also be built with AddressSanitizer, UndefinedBehaviorSanitizer, and warnings as errors. An append-only std::vector owns the full records. Both indexes implement IIndex and store only a trade ID and a zero-based row position. The position stays valid when the vector reallocates. Lookups return an optional position so that row zero and a missing ID are distinct. The C++ standard library provides vectors, smart pointers, binary search, pseudorandom generation, file streams, and std::chrono::steady_clock for timing.

Hash table. The hash table starts with 16 buckets. A 64-bit integer mixer spreads keys across the buckets. A vector within each bucket holds colliding entries, which are searched by exact ID. Before an insertion would push the load factor above 0.75, the implementation doubles the bucket count and redistributes the entries.

B+ tree. The default maximum is 32 children per internal node. Binary search over separators chooses a child, with equal keys routed to the right. A second binary search locates the ID in the leaf. Leaves hold sorted keys and their row positions, and a link points to the next leaf. An overflowing leaf splits and copies its right half's first key into the parent. An overflowing internal node promotes a separator. Splits can propagate to the root and create a new root, keeping all leaves at the same depth.

Benchmark procedure. For each size and order, every trial builds an empty index (timed), runs 100,000 successful and 100,000 unsuccessful lookups on that index (timed), and then inserts max(1, n/10) additional keys (timed). Unsuccessful lookups use random absent IDs that lie between stored keys, so they visit all parts of the tree; if a dataset has no gaps the runner falls back to IDs above every stored key. Both indexes receive identical keys and alternate which runs first between trials. Each configuration uses one warmup and five measured trials by default. The runner verifies every returned position, compares checksums between the two indexes, and checks index structure and all mappings after the timed regions. The benchmark runs independently of the active database.

Distribution of responsibility. The proposal assigned Zhengmao Zhang the hash table, collision and resize testing, and hash benchmark integration. Jacob Lindland was assigned the B+ tree, its insertion and split tests, and tree benchmark integration. My Pham was assigned record storage, data generation, CSV I/O, the menu, and the benchmark runner. The repository records Jacob's implementation of the B+ tree and its Catch2 tests, and Zhengmao's integration of the CLI, database, and benchmark components. The final revision added tests for the hash index, record validation, CSV I/O, database, dataset generator, benchmark runner, and menu.

2. Analysis

Changes from the proposal. The proposal described generating synthetic CSV datasets. The benchmark menu generates its datasets directly in memory, which lets users run the suite without preparing input files and keeps CSV parsing outside the measurements. The menu focuses on exact-ID lookup so the comparison uses the same operations on both indexes. The B+ tree retains linked leaves for a possible range-query extension. CSV export follows the design's write-then-rename approach and uses a unique staging directory so separate saves cannot share a temporary file.

Benchmark revision. Our first benchmark run drew absent IDs only from above the largest stored ID. Every such lookup followed the rightmost path of the tree, which stays in cache, so the B+ tree appeared to answer misses in 12–15 ns while its hits took 90–160 ns. We changed the generator to store odd IDs and draw absent IDs from the even values between them, and we rebuilt the runner so each trial builds an index once and runs both lookup workloads on it before the timed inserts. All numbers below come from the revised benchmark.

Worst-case running time. Let n be the number of indexed records, m the number of rows imported, b the number of bytes in the input file, and q the number of lookups in a batch. Assume a fixed B+ tree node capacity and the default hash-table configuration.

Operation	Worst case	Reason
Hash lookup	O(n)	One bucket can contain all keys.
Hash insertion	O(n)	Scanning a chain or rehashing can take linear time.
B+ tree lookup	O(log n)	Binary search at each level of a balanced tree.
B+ tree insertion	O(log n)	A split may propagate up one root-to-leaf path.
Database insertion	O(n)	Hashing or growing the record vector can take linear time.
CSV load	$O(b + m^2)$	Read and parse the file, then build both indexes.
CSV save	O(n)	Validate and write each record once.

Dataset generation	O(n)	Generate each row and optionally shuffle the vector.

Database lookup inherits the selected index's bound because resolving the row position and copying a bounded-size record takes constant time. A well-distributed hash gives expected O(1) lookup and expected amortized O(1) insertion. Building from empty takes $O(n^2)$ in the worst case for hashing and O(n log n) for the B+ tree. An incremental insert batch adds linearly many keys and has the same bounds. The timed lookup batches take O(qn) and O(q log n), respectively. Outside timing, the hash structure check can also take $O(n^2)$ because it scans earlier entries in each bucket for duplicates. The record store and both indexes together use O(n) space under the default configuration.

Measured results. Each cell shows hash / B+ tree median nanoseconds per operation over five measured trials. Build time is divided by the number of records built; insert time by the number of added keys (10% of the dataset). The run used a Release build (Apple clang 21.0.0, -O3 -DNDEBUG) on an Apple M5 Mac with 10 logical threads running macOS, with seed 3530 and 100,000 lookups per batch; the raw trials are in results/submission-benchmark.csv.

Rows	Order	Build	Insert	Hit	Miss
100,000	Sequential	76.65 / 33.99	22.69 / 42.55	5.59 / 36.28	6.24 / 33.44
100,000	Shuffled	74.88 / 100.17	23.10 / 96.71	5.80 / 46.41	6.28 / 45.09
250,000	Sequential	80.05 / 55.06	33.53 / 61.38	9.65 / 50.19	8.88 / 40.77
250,000	Shuffled	78.05 / 111.48	33.38 / 103.31	9.14 / 56.43	8.79 / 55.22
500,000	Sequential	89.39 / 49.51	46.25 / 66.06	12.47 / 65.99	10.94 / 50.55
500,000	Shuffled	92.79 / 126.05	50.21 / 118.94	12.77 / 69.42	12.48 / 66.86

Trial-to-trial variation is visible in the raw data: the gap between a cell's fastest and slowest trial is up to about 25% of its median for builds and B+ tree lookups, up to 44% for inserts and 34% for hash hits, and one slow outlier trial makes the spread of a hash miss cell 72%. However, for all 24 combinations of size, order, and workload, the five hash trials and the five B+ tree trials do not overlap, so every comparison below holds for the fastest and slowest trial alike.

Interpretation. The hash table answered lookups 4.6 to 8.0 times faster than the tree in all twelve successful and unsuccessful configurations (5.6–12.8 ns against 33–69 ns). Hash lookups touch one bucket, whereas a tree lookup descends several levels and binary-searches each node, and the tree's cost grows with n as its nodes stop fitting in cache (36 to 66 ns per hit for sequential data). With misses spread across the key range, tree misses cost about as much as hits, which confirms that the first run's 13 ns figure was an artifact of the out-of-range workload. Tree lookups were slightly faster on sequentially built trees than on shuffled ones (for example 36 against 46 ns per hit at 100,000 rows); a plausible reason is that nodes allocated in key order sit closer together in memory, though we did not measure cache behavior directly.

The B+ tree built faster than the hash table only for sequential data (34, 55, and 50 ns per key against 77, 80, and 89). Increasing IDs send every tree insert to the rightmost leaf, which appends without shifting entries. Shuffled inserts reach different leaves and shift entries within them, and the hash table was faster for all three shuffled builds (75–93 ns against 100–126). The hash build is much more expensive per key than a hash insert into an existing table (75–93 against 23–50 ns). We checked why with a separate run: growing the table from 16 buckets and rehashing at every doubling roughly doubled the build cost, since building 500,000 keys took about 100–118 ns per key from 16 buckets but about 48–50 ns per key when the table started with 2^{20} buckets. The reported hash build times therefore include the growth policy rather than only the cost of hashing and inserting. The 10% batch of additional keys was inserted faster by the hash table in all six configurations (23–50 ns against 43–119), because the table does not need to grow again at these sizes (the bucket count is the same before and after the batch in the raw data), while every B+ tree insert pays a binary search and, in shuffled order, entry shifts in the leaf.

3. Reflection

Project experience and challenges. The project combined implementing the indexes with designing a way to compare them. The shared interface made it possible to use one benchmark for both implementations. The B+ tree exposed a subtle correctness issue to test: when a leaf splits, searching for the promoted separator must still find the record in the leaf on the right. Small node capacities made it possible to force leaf, internal, and root splits with manageable test sequences, and randomized comparisons against a reference map gave the same confidence for both indexes. Another challenge was distinguishing an indexing failure from an input failure. Keeping the old database until a CSV import has passed validation gave the application a clear rule for handling malformed rows and duplicate IDs. The most instructive problem was the benchmark itself: a plausible-looking result for missing keys was an artifact of where the keys were drawn, and we only found it by asking which tree nodes each workload touched.

What we would change. We would add shared index tests and a CSV-to-lookup smoke test early so every component has the same acceptance criteria, rather than extending the suite at the end. We would archive each benchmark run with its source revision as well as its build settings, and add a pre-sized hash variant to separate growth cost from hashing cost. We would also use the linked leaves to add range queries, which is the main reason to prefer a B+ tree over a hash table and would complete the comparison.

What each member learned. *Zhengmao Zhang* (hash table and integration) learned that a chained hash table's speed depends on its growth policy as much as on its hash function, and that integrating the database, menu, and benchmark requires every component to honor the same row-position contract. *Jacob Lindland* (B+ tree) learned to state the tree's invariants precisely, such as equal keys routing right and separators being copies of leaf keys, and to check them after every insert with a structural validator and small node sizes. *My Pham* (storage, data generation, CSV, menu, and benchmark runner) learned that benchmark design shapes the result: where absent keys are drawn from, what is built once or per trial, and what is verified outside timing all changed the conclusions, and that scripted menu sessions make the interface testable. As a team we also confirmed the value of agreeing on the architecture early, assigning clear ownership, and communicating as work progresses, since both indexes had to return the same row position for the same trade ID.

References

[1] Oracle (Group 94), *MiniDB Database Indexing Proposal*.

[2] MiniDB, *Component Design and Interface Specification*. docs/MiniDB_Design.md.

[3] MiniDB, implementation and tests.

[4] MiniDB, archived benchmark trials. results/submission-benchmark.csv.